Estimating The Energy Consumption of Quantum Computing from A Full System Aspect

Siyuan Niu
University of Central Florida
Orlando, Florida, USA
siyuan.niu@ucf.edu

Di Wu
University of Central Florida
Orlando, Florida, USA
di.wu@ucf.edu

Ozgur Ozan Kilic
Brookhaven National Laboratory
Upton, New York, USA
okilic@bnl.gov

Kwangmin Yu
Brookhaven National Laboratory
Upton, New York, USA
kyu@bnl.gov

Abstract

Quantum computing promises disruptive capabilities, yet its energy footprint has received far less attention than its asymptotic speedups. We present a first-order, full-system energy model for quantum computing in an high performance computing (HPC) context. The model separates costs common to NISQ and FTQC, such as system maintenance and classical processing, from regime-specific ones such as error mitigation for NISQ and error correction for FTQC. We instantiate the model on 96- and 100-qubit Heisenberg time-evolution simulations on IBM Eagle r3 and a representative VQE workload, and sketch the FTQC energy pipeline. We find that NISQ energy is dominated by the QEM sampling multiplier, while FTQC cost shifts to physical-qubit overhead set by the code distance and magic states. Our model provides actionable insights into the energy consumption of both NISQ and FTQC workloads, and paves the way toward energy-efficient quantum advantage.

Keywords: energy consumption, noisy intermediate-scale quantum, fault tolerant quantum computing

1 Introduction

Quantum computing offers disruptive computing capabilities, such as prime factorization and combinatorial optimizations, quantum simulation and cryptography, which are beyond the reach of classical computing. With the recent improvements on both quantum algorithms and hardware, prototyping utility-scale quantum computers is on the horizon. Despite tremendous efforts in the computing capability, such as functional correctness and performance, one untouched research question of quantum computing is: *how much energy does quantum computing consume?* This question is critical to energy, cost, and power grid allocation in high performance computing (HPC) systems, where quantum computers are integrated as important sub-components. Understanding the energy landscape will answer a pivotal question: *when and where to apply quantum computing*.

However, quantum energy estimation is complicated by the heterogeneity of the environment in which quantum algorithms are executed. Quantum processors are rarely operated in isolation; they increasingly act as accelerators inside HPC-cloud-quantum substrates, in which each subtask is, in principle, routed to its most efficient compute resource. The prototype of such substrates have already been demonstrated in industries, such as the quantum-centric supercomputing from IBM [28] and NVQLink from Nvidia [5]. Energy accounting across such substrates is fragmented across siloed mechanisms: facility power-usage effectiveness (PUE) metrics report infrastructure efficiency without resolving job-level consumption [2]; node-level counters capture only partial IT loads [11]; cloud billing approximates resource usage rather than energy [10]; and quantum services seldom expose cryogenic or control overheads [14]. Consequently, no unified cost model exists that simultaneously accounts for classical data movement, network transfer, and quantum-specific contributions such as cryogenic cooling and control electronics.

Existing quantum-energy models [14, 24] address only a subset of these concerns. First, they rely on quantum-volume worst-case estimation, which overstates energy consumption by neglecting recent architectural advances such as dynamic circuits with mid-circuit measurement, feed-forward control, and conditional execution [25, 29], all of which reduce the effective circuit size. Second, they are formulated for a single qubit technology, typically superconducting, and do not generalize to trapped-ion, photonic, or neutral-atom platforms, including maintenance budgets for ultra-high vacuum, cryogenic cooling, and laser stabilization that differ by orders of magnitude. Third, they restrict their scope to the quantum subsystem and omit the classical pre- and post-processing on which every realistic workload depends.

The energy footprint of quantum computation is therefore determined by three interacting factors: the *quantum application*, which fixes gate count, circuit depth, and qubit count; the *quantum hardware*, which fixes per-gate energy and maintenance overhead through the underlying qubit technology; and the *operating regime* under which the device executes. In this work, we focus on both noisy intermediate-scale quantum (NISQ) computing and fault-tolerant quantum computing (FTQC) regimes.

The energy profiles for NISQ and FTQC are different. NISQ devices tolerate hardware imperfection and rely on quantum error mitigation (QEM), comprising techniques such as zero-noise extrapolation, Pauli twirling, dynamical decoupling, and measurement mitigation [8, 9, 21, 22, 32]. QEM introduces no qubit overhead but inflates circuit and shot counts; the dominant energy cost therefore arises from additional circuit execution and repeated sampling. FTQC, in contrast, suppresses noise through QEC. The associated qubit overhead is substantial depending on the QEC code, and is accompanied by continuous syndrome measurement, real-time classical decoding, and magic-state distillation/cultivation [3, 18]. In this regime, energy consumption is dominated by physical-qubit maintenance, decoder hardware, and the multiplicative overhead of logical operations. Although both regimes share a common substrate (gate execution, qubit idling, system maintenance, and classical processing), they differ in their treatment of error, which gives rise to distinct optimization strategies and distinct projections as the technology scales.

The contribution of the paper is listed as follows:

- We summarize the energy sources of a quantum workload, separating costs common to NISQ and FTQC from regime-specific overheads.
- We quantify the NISQ side with two case studies: 96- and 100-qubit Heisenberg time-evolution simulations on IBM Eagle r3 hardware, and a representative VQE workload.
- We sketch the FTQC energy pipeline and show how the dominant cost shifts from sampling to physical-qubit overhead.

2 Where Does the Energy Come From?

A quantum workload draws energy from several distinct sources. We separate them into a small *common overhead* that is paid by both NISQ and FTQC executions, and a *regime-specific execution* term that captures the algorithm-dependent cost of running the workload under either QEM or QEC.

2.1 Common Overhead in NISQ and FTQC

System maintenance. Different qubit technologies impose drastically different maintenance budgets: superconducting qubits require dilution refrigerators that draw ~10-25 kW continuously; trapped ions require ultra-high-vacuum chambers and stabilized lasers; photonic qubits require single-photon sources and detectors with their own cooling. This term is technology-dependent and largely independent of the specific algorithm being run, which makes it a consequential denominator when comparing platforms [24].

Classical pre/post-processing. Every quantum job is flanked by classical work: circuit transpilation/compilation, parameter optimization, results aggregation, and integration with HPC pipelines (file I/O, network transfer, scheduler overhead). For hybrid HPC-cloud-quantum substrates, this can be expressed as a job-level energy boundary

$$E_{\mathrm{cls}} = \mathrm{PUE}(t) \cdot E_{\mathrm{IT}} + E_{\mathrm{shared}} + E_{\mathrm{net,WAN}} + E_{\mathrm{storage}}, \quad (1)$$

with $\mathrm{PUE}(t)$ the time-varying facility efficiency [34], E_{IT} aggregating per-node package/DRAM/GPU/NIC energy via RAPL/NVML and equivalent counters [10, 11], and the remaining terms covering shared infrastructure, network, and storage.

2.2 NISQ-Specific Overhead

Gate execution. Every quantum operation has a non-zero energy-per-gate E_g, dependent on qubit technology. Reported values include $E_g \approx 0.18$ J for superconducting two-qubit gates and $E_g \approx 15$ J for trapped-ion two-qubit gates [14], both inclusive of system cooling overhead. However, gate-energy estimation remains an early-stage research area, and reported values vary substantially across studies and methodologies [20, 30].

The bare per-circuit physical-gate energy is

$$E_{\mathrm{gate}} = \sum_g E_g \cdot N_g, \quad (2)$$

summed over all gate types g with their counts N_g in the executed circuit.

Error mitigation. QEM accepts imperfection of near-term quantum devices and adopts methods of mitigating or suppressing quantum noises [13, 32]. The cost mainly comes from sampling and circuits. We first isolate the baseline sampling cost that any NISQ workload incurs, and then quantify the QEM-specific multipliers that act on top of it. A single quantum circuit returns one bitstring per execution, so estimating an expectation value $\langle H\rangle$ to statistical precision ϵ requires $S \sim O(1/\epsilon^2)$ shots, typically $S \in [10^3, 10^5]$. This sampling cost is intrinsic to quantum measurement; it is present even in the absence of any mitigation, so we treat S as a baseline rather than as a QEM mechanism. On top of this baseline, we analyze the overhead of two QEM techniques:

- **Zero-Noise Extrapolation (ZNE).** The same circuit is executed at multiple effective noise levels (e.g., scaling factors $\{1, 3, 5\}$ via gate folding), giving a multiplicative factor $\sum_k \alpha_k$ on the raw gate count, where α_k is the fold of copy k.
- **Pauli Twirling (PT).** Each ZNE copy is replicated P times with random Pauli insertions, which converts coherent error into a stochastic Pauli channel and stabilizes the ZNE extrapolation. This contributes a multiplicative factor P on top of ZNE.

Combining the baseline sampling cost with the two QEM multipliers, the NISQ-specific execution energy is therefore

$$E^{\mathrm{exec}}_{\mathrm{NISQ}} = \Big(\sum_k \alpha_k\Big) \cdot P \cdot S \cdot E_{\mathrm{gate}} \quad (3)$$

Other error mitigation techniques like dynamical decoupling (DD) and Matrix-free Measurement Mitigation (M3) do not duplicate the full circuit, so they do not enter Eq. (3) as multiplicative factors, but they are not free. DD inserts pulse sequences into idle windows; this incremental gate cost is absorbed into E_{gate}. M3 requires a one-time calibration of the readout assignment matrix A on the relevant measurement subspace, consuming S_{cal} shots that contribute to E_{gate} but are amortized over subsequent evaluations; the sparse classical inversion of A is folded into E_{cls} in Eq. 1.

2.3 FTQC-Specific Overhead

FTQC encodes a logical qubit into many physical qubits and detects errors via continuous syndrome measurement. Different QEC code families exhibit different code rates and span across wide code distances. Despite of the differences, three energy-relevant cost centers result:

- **Encoding overhead.** We focus on the most explored surface code, which requires $\Theta(d^2)$ physical qubits per logical qubit at distance d. Each logical gate is implemented through a sequence of physical operations that scales with d, multiplying E_{gate} accordingly. The logical error rate of surface code scales as $p_L \propto (p/p_{\text{th}})^{(d+1)/2}$. With a threshold of $p_{\text{th}} \approx 1\%$, a target logical error rate of 10^{-12}, and a physical error of 10^{-3}, the surface code needs $d \approx 25$ [15, 17], implying roughly 10^3 physical qubits per logical qubit. The encoding cost will be lower for qLDPC codes. The unit of a logical circuit is one spacetime cell whose energy is denoted by E_{cyc} and the total spacetime volume of the compiled program is V_{ls}.
- **Real-time decoding.** Classical decoders process syndrome information and produce corrections inside a strict latency budget which is on the order of μs for superconducting devices [1]. Further considering the syndrome measurement to avoid backlog problem [33], decoding has to complete within $400ns$ for superconducting-based quantum computers [16]. Moreover, decoding cost grows with code distance and varies with implementation (CPU, FPGA, ASIC) and physical placement (cold, 4 K stage, or room-temperature) [14].
- **Magic-state distillation/cultivation.** Non-Clifford operations (e.g., T-gates) are not transversal in most codes and must be realized using prepared high-fidelity magic states. Conventional distillation factories consume large numbers of physical qubits and many code cycles per distilled state [23]; more recent magic-state cultivation protocols grow T states in place and reduce the qubit overhead significantly but remains sampling overhead [18]. Either approach contributes a separate E_{ms} term whose share grows with T-gate count.

The FTQC-specific energy is therefore

$$E_{\text{FTQC}}^{\text{exec}} = V_{\text{ls}} \cdot E_{\text{cyc}} + N_T \cdot E_{\text{ms}} + E_{\text{dec}} \tag{4}$$

where V_{ls} is the total space time volume of the compiled logical circuit, E_{cyc} is the energy of a spacetime cell, N_T is the number of magic states, and E_{dec} is the decoding energy.

Putting it together. For a given quantum workload, the total energy is

$$E_{\text{tot}} = \underbrace{E_{\text{sys}} + E_{\text{cls}}}_{\text{common overhead}} + \underbrace{E_{\text{NISQ}}^{\text{exec}} \text{ or } E_{\text{FTQC}}^{\text{exec}}}_{\text{regime-specific execution}} \tag{5}$$

with at most one of the two regime-specific terms active. NISQ workloads spend most of their joules on the QEM multiplier; FTQC workloads spend joules evenly.

3 Case Studies on NISQ Algorithms

We instantiate the common and NISQ-specific terms of Eq. (3) on two workloads of current scientific interest: Hamiltonian time evolution and the variational quantum eigensolver.

3.1 Time-Evolution Simulation of a Heisenberg Spin Chain

Setup. We benchmark the time evolution of the isotropic Heisenberg Hamiltonian H_{iso} on IBM Eagle r3 superconducting processors, extending the experimental setup of [9] to large-scale noisy quantum hardware and adding an end-to-end energy-consumption estimate. Two configurations are evaluated: N=100 qubits with open boundary conditions (OBC) on `ibm_brisbane`, and N=96 qubits with periodic boundary conditions (PBC) on `ibm_sherbrooke`. The PBC count is reduced from 100 to 96 in order to permit a coupling between the first and last qubits.

Quantum-error-mitigation stack. QEC is impractical at this scale on contemporary superconducting hardware due to its qubit overhead, so we instead rely on QEM, which accepts hardware imperfection at the cost of an additional circuit duplication and sampling. Four QEM techniques are stacked: ZNE with scaling factors $\{1, 3, 5\}$ (three folding copies per circuit); PT with $P = 10$ randomized copies per fold; DD inserted into idle windows; and M3 for readout-error correction. Each circuit is measured at $S = 10^5$ shots. Substituting these settings into Eq. (3) gives $E_{\text{QEM}} = (1+3+5) \cdot P \cdot S \cdot E_g \cdot N_{\text{gates}}$, where N_{gates} is the post-transpile gate count.

Energy estimate. Following the per-technology energy-per-gate model of [14], we adopt $E_g \approx 0.18$ J for superconducting two-qubit gates (compared with ~ 15 J for trapped-ion gates), which implicitly absorbs operational and cooling overhead into the gate-energy term. The total experiment energy is obtained by counting all gates across the duplicated circuits of the QEM stack (Qiskit transpile, optimization level 3) and multiplying by the shot count $S = 10^5$ shots as presented in Table 1.

	OBC (100 sites)			PBC (96 sites)		
ZNE fold	1	3	5	1	3	5
No. of gates with PT	24,040	49,300	74,560	25,400	49,880	74,360
Energy (kJ) with PT	432,720	887,400	1,342,080	457,200	897,840	1,338,480
Total Energy (kJ)	2,662,200			2,693,520		
Power (MW)	2.686			2.684		

Table 1. The total energy consumption of the Hamiltonian simulation. The total QPU time of the simulations is 16 minutes 44 seconds and 16 minutes 31 for PBC and OBC, respectively. Circuit depth with respect to the Trotter steps for H_{iso} with $N = 96, 100$ qubits for PBC and OBC, respectively, after Qiskit transpile with the optimization level 3.

3.2 Variational Quantum Eigensolver (VQE)

VQE estimates ground-state energies of a Hamiltonian $H = \sum_p c_p P_p$ (decomposed into Pauli strings P_p) by variationally minimizing $\langle\psi(\boldsymbol{\theta})|H|\psi(\boldsymbol{\theta})\rangle$ over a parameterized ansatz $|\psi(\boldsymbol{\theta})\rangle$. Its energy footprint differs from time evolution due to the short Ansatz and the iterative algorithm.

Energy model. For a workload with K optimizer iterations, M groups of Pauli strings to estimate per iteration (where one group corresponds to one circuit), S shots per circuit, and an Ansatz of G two-qubit gates, the gate cost is

$$E_{\text{gate}}^{\text{VQE}} = G \cdot M \cdot S \cdot K \cdot E_g. \tag{6}$$

QEM (e.g., ZNE+PT) multiplies this further by $(\sum_k \alpha_k) \cdot P$ as in Eq. (3). The classical optimizer, Hamiltonian grouping, and parameter update contribute to E_{cls} and become non-trivial when K is large or the optimizer is non-trivial.

Comparison. The two NISQ workloads show that VQE energy is dominated by the optimizer-loop multiplier K, whereas time-evolution energy is dominated by circuit depth and the per-fold ZNE multiplier. Improvements that reduce K such as better warm-starts [37], adaptive ansatze [19] can reduce the energy consumption of VQE; whereas improvements that reduce QEM overhead such as better-chosen ZNE scale factors [4] can help with energy consumption for time evolution.

4 Energies of FTQC: A Pipeline View

The FTQC era inherits the common overhead of Eq. (5) and pays $E_{\text{FTQC}}^{\text{exec}}$ from Eq. (4) on the regime-specific side. We analyze the end-to-end compilation pipeline that maps a fault-tolerant quantum algorithm down to physical qubit-cycles, and detail the energy consumption of each compilation stage below.

Stage 1: Fault-tolerant quantum algorithm to logical quantum circuit. A fault-tolerant quantum algorithm (e.g., Shor, Grover, quantum chemistry, Hamiltonian simulation) is synthesized to a logical circuit with a fault-tolerant gate set. These sets typically consist of a non-Clifford gate combined with the Clifford group, with the Clifford+T set being a standard choice. This synthesis process can be implemented using existing tools such as AlphaTensor [27]. The circuit includes N_L logical qubits, N_T T-gates, and N_C Clifford gates. This stage sets N_T in Eq. (4).

Stage 2: Encoding into physical qubits. We utilize the well explored surface code for the compilation pipeline and employ lattice surgery for logical operations, as this approach is compatible with a wide range of quantum platforms, including limited 2D nearest-neighbor architectures for superconducting qubits and more flexible architectures of neutral atoms. Although transversal Clifford gates achieve $\Theta(1)$ time complexity, their implementation is contingent upon specific hardware capabilities, such as 3D inter-patch coupling or atom-style qubit transport [7]. The energy of a logical qubit cell E_{cyc} is fixed by the physical platform together with the chosen code distance d, which is in turn picked to meet the target logical error rate $p_L \approx (p/p_{\text{th}})^{(d+1)/2}$ over the workload's spacetime volume. This stage sets E_{cyc} in Eq. (4).

Stage 3: Logical-circuit compilation. The Clifford+T program of Stage 1 must be scheduled into a concrete spacetime layout of data patches, magic-state factories, ancilla bus channels, and merge/split operations. This can be achieved by open-source lattice surgery compilers such as TQEC [31]. The compiled layout determines the spacetime volume, which directly multiplies the energy cost of one logical qubit cell in Stage 2, and the end-to-end energy estimates can inherit the quality of the schedule. This stage sets V_{ls} in Eq. (4).

Stage 4: Magic-state distillation/cultivation. T-gates are realized by consuming high-fidelity $|T\rangle$ states prepared offline. Conventional magic-state distillation protocols achieve qubit cost $\Theta(d^2 \cdot k)$ for k levels of distillation [23]. For example, a single-level 15-to-1 factory at distance $d_f \approx 15$ produces output states with error $\sim 10^{-8}$ at roughly 810 qubit-cycles per output T [23]. More recent magic-state cultivation [18] grows a T state directly inside a small surface-code patch and then expands the patch fault-tolerantly to the target distance, reducing spacetime volume per T by roughly an order of magnitude relative to distillation at comparable output fidelity. This stage sets E_{ms} in Eq. (4).

Stage 5: Real-time decoding. Each surface-code cycle produces $\Theta(d^2)$ syndrome bits per logical patch that must be decoded inside the cycle latency. Reference implementations can place decoders in the cryostat or at room temperature.

Each placement trades off accuracy, latency, energy, and thermal load on the dilution refrigerator. Quantum system designers must carefully choose the decoder to meet all system requirements while minimizing the energy consumption. Our hardware implementation of a BPOSD decoder [26] indicates a median decoding latency of $25ns$ for $d = 11$, while a minimum weight perfect matching (MWPM) decoder [35] shows $33ns$ with similar logical error rates at $4.6\times$ higher power. Going to $d = 32$, MWPM needs $12.2\times$ higher power. The gap will grow exponentially larger if scaling the decoding system to more logical qubits, due to low scalability of MWPM [12]. This stage sets E_{dec} in Eq. (4). We show a collection of our implementations [36] in Table 2.

Table 2. Average metrics of BPOSD (B) and MWPM (M) hardware decoders for one logical qubit across distances.

d	**Area** (mm^2)		**Power** (W)		**Latency** (ns)	
	B	**M**	**B**	**M**	**B**	**M**
7	0.90	0.38	0.27	0.19	19.6	14.4
11	1.62	1.76	0.28	0.92	26.6	35.5
13	4.35	3.10	0.36	1.62	32.8	49.6
32	57.45	59.09	2.49	30.33	145.0	300.5

Stage 6: System maintenance at scale. Cryogenic load grows with physical-qubit count. A 1M-qubit superconducting machine implies multiple parallel dilution refrigerators and tens to hundreds of kilowatts of continuous power, dwarfing the per-job dynamic energy [24]. This stage feeds E_{sys} in Eq. (1), not $E^{\text{exec}}_{\text{FTQC}}$.

Where the joules go. In logical quantum circuits, $E_{\text{ms}} \sim 10^3\text{-}10^4 E_{\text{cyc}}$ for distillation, $\sim 10\times$ less for cultivation [18, 23]. T-heavy workloads (Shor, chemistry) are bound by $N_T E_{\text{ms}}$; Clifford-heavy workloads by $V_{\text{ls}} E_{\text{cyc}}$; both pay E_{sys} as a continuous cryogenic tax that, at million-qubit scale, runs at $\sim$0.1-1 MW unconditionally [24].

Implications. The crossover question is whether classical alternatives consume more energy than the FTQC version of the same algorithm. Prior work [6] suggests that quantum acceleration must exceed $\sim 100\times$ to offset cryogenic overheads alone, since dilution refrigerators operating near 10 mK draw tens of kilowatts of wall-plug power regardless of the computational workload they support. Eq. (4) makes this concrete: the inequality is a comparison of measurable quantities in joules, not gate counts. This reframing has several consequences worth drawing out. First, the dominant term tells the architect where to push: lowering E_{ms} through better magic-state protocols pays off for T-heavy workloads, while shrinking V_{ls} through tighter scheduling and lower code distance pays off for Clifford-heavy ones, and effort spent on the wrong lever yields negligible returns. Second, E_{dec} deserves attention disproportionate to its apparent size in the equation: decoding runs continuously on classical hardware throughout the computation, and if it cannot keep pace with the syndrome extraction rate, the logical clock stalls and the cryostat continues to draw power without producing useful work. A slow decoder thus inflates E_{sys} multiplicatively rather than adding to the budget linearly, which makes decoder throughput and energy efficiency a leveraged investment, improvements there reduce both E_{dec} itself and the idle tax it would otherwise impose. Third, E_{sys} behaves qualitatively differently from the operation-based terms: it is paid in wall-clock time rather than in gates, so any calibration pause, serial bottleneck, or classical-feedback latency inflates the total energy without producing computation. This favors algorithms that keep the device saturated and penalizes those with sparse parallelism. Finally, energy accounting closes a loophole that gate-count comparisons leave open: a quantum algorithm with a favorable asymptotic speedup can still lose on joules if its prefactor is dominated by magic-state synthesis, if its decoder bottlenecks the logical clock, or if its runtime forces the cryostat to absorb a large idle tax. The practical suggestion is that algorithm designers report a joule budget alongside gate counts, and that hardware roadmaps quote E_{cyc}, decoder throughput per watt, and refrigerator wall-plug efficiency as first-class figures of merit, since these are the quantities that actually determine whether a given application crosses the classical threshold.

5 Conclusion

Given the rise of quantum utility, it is critical to understand its energy consumption, a critical metric that has become a bottleneck in the AI era. We separated the energy sources of a quantum workload into those common to NISQ and FTQC and those specific to each regime, instantiated the NISQ side on 96- and 100-qubit Heisenberg time-evolution experiments and a representative VQE chemistry workload, and traced the FTQC pipeline from logical circuit through magic-state distillation/cultivation to syndrome decoding. The case studies show that error-handling overhead, not gate count, not cryogenics, sets the energy budget today, and that the dominant cost will shift from sampling multipliers in the NISQ regime to encoding and magic state in the FTQC regime. Making these costs explicit is a prerequisite for deciding when and where quantum computing should be applied. For future works, we plan to demonstrate an end-to-end energy-consumption pipeline for fault-tolerant quantum algorithms, paving the road toward efficient quantum advantage.

References

[1] 2025. Quantum error correction below the surface code threshold. *Nature* 638, 8052 (2025), 920–926.

[2] Victor Avelar, Dan Azevedo, and Alan French. 2012. *PUE: A Comprehensive Examination of the Metric*. Technical Report. The Green Grid.

[3] Sergey Bravyi and Jeongwan Haah. 2012. Magic-state distillation with low overhead. *Physical Review A—Atomic, Molecular, and Optical Physics* 86, 5 (2012), 052329.

[4] Zhenyu Cai, Ryan Babbush, Simon C Benjamin, Suguru Endo, William J Huggins, Ying Li, Jarrod R McClean, and Thomas E O'Brien. 2023. Quantum error mitigation. *Reviews of Modern Physics* 95, 4 (2023), 045005.

[5] Shane A Caldwell, Moein Khazraee, Elena Agostini, Tom Lassiter, Corey Simpson, Omri Kahalon, Mrudula Kanuri, Jin-Sung Kim, Sam Stanwyck, Muyuan Li, et al. 2025. Platform Architecture for Tight Coupling of High-Performance Computing with Quantum Processors. *arXiv preprint arXiv:2510.25213* (2025).

[6] S. Chen. 2023. Are Quantum Computers Really Energy Efficient? *Nature Computational Science* 3 (2023), 457–460.

[7] Zi-Han Chen, Ming-Cheng Chen, Chao-Yang Lu, and Jian-Wei Pan. 2026. Transversal logical Clifford gates on the rotated surface code with reconfigurable neutral atom arrays. *Physical Review Letters* 136, 13 (2026), 130601.

[8] Seokwon Choi, Talal Ahmed Chowdhury, and Kwangmin Yu. 2025. Quantum Utility-Scale Error Mitigation for Quantum Quench Dynamics in Heisenberg Spin Chains. *arXiv e-prints* (2025). arXiv:2506.20125 [quant-ph] https://arxiv.org/abs/2506.20125

[9] Talal Ahmed Chowdhury, Kwangmin Yu, Mahmud Ashraf Shamim, M. L. Kabir, and Raza Sabbir Sufian. 2024. Enhancing quantum utility: Simulating large-scale quantum spin chains on superconducting quantum computers. *Physical Review Research* 6, 3, Article 033107 (July 2024), 033107 pages.

[10] Mehiar Dabbagh, Bechir Hamdaoui, Mohsen Guizani, and Ammar Rayes. 2014. Energy-Efficient Cloud Resource Management. In *IEEE INFOCOM Workshops*.

[11] Howard David, Eugene Gorbatov, Ulf Hanebutte, Rahul Khanna, and Christian Le. 2010. RAPL: Memory Power Estimation and Capping. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*.

[12] Nicolas Delfosse and Naomi H Nickerson. 2021. Almost-linear time decoding algorithm for topological codes. *Quantum* 5 (2021), 595.

[13] Suguru Endo, Simon C. Benjamin, and Ying Li. 2018. Practical Quantum Error Mitigation for Near-Future Applications. *Physical Review X* 8, 3, Article 031027 (July 2018), 031027 pages. arXiv:1712.09271 [quant-ph] doi:10.1103/PhysRevX.8.031027

[14] Rolando P Hong Enriquez, Rosa M Badia, Barbara Chapman, Kirk Bresniker, Aditya Dhakal, Eitan Fractenberg, Gourav Rattihalli, Ninad Hogade, Pedro Bruel, Alok Mishra, et al. 2023. Estimating energy-efficiency in quantum optimization algorithms. In *Cray User Group Conference Proceedings*.

[15] Austin G Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. 2012. Surface codes: Towards practical large-scale quantum computation. *Physical Review A—Atomic, Molecular, and Optical Physics* 86, 3 (2012), 032324.

[16] Joydip Ghosh, Austin G Fowler, and Michael R Geller. 2012. Surface code with decoherence: An analysis of three superconducting architectures. *Physical Review A—Atomic, Molecular, and Optical Physics* 86, 6 (2012), 062318.

[17] Craig Gidney and Martin Ekerå. 2021. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* 5 (2021), 433.

[18] Craig Gidney, Noah Shutty, and Cody Jones. 2024. Magic state cultivation: growing T states as cheap as CNOT gates. *arXiv preprint arXiv:2409.17595* (2024).

[19] Harper R Grimsley, Sophia E Economou, Edwin Barnes, and Nicholas J Mayhall. 2019. An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nature communications* 10, 1 (2019), 3007.

[20] Daniel Jaschke and Simone Montangero. 2023. Is quantum computing green? An estimate for an energy-efficiency quantum advantage. *Quantum Science and Technology* 8, 2 (2023), 025001.

[21] Abhinav Kandala, Kristan Temme, Antonio D. Córcoles, Antonio Mezzacapo, Jerry M. Chow, and Jay M. Gambetta. 2019. Error mitigation extends the computational reach of a noisy quantum processor. *Nature* 567, 7749 (March 2019), 491–495. arXiv:1805.04492

[22] Youngseok Kim, Christopher J. Wood, Theodore J. Yoder, Seth T. Merkel, Jay M. Gambetta, Kristan Temme, and Abhinav Kandala. 2023. Scalable error mitigation for noisy quantum circuits produces competitive expectation values. *Nature Physics* 19, 5 (Feb. 2023), 752–759.

[23] Daniel Litinski. 2019. Magic state distillation: Not as costly as you think. *Quantum* 3 (2019), 205.

[24] Michael James Martin et al. 2022. Energy use in quantum data centers: Scaling the impact of computer architecture, qubit performance, size, and thermal parameters. *IEEE Transactions on Sustainable Computing* (2022).

[25] Siyuan Niu, Efekan Kokcu, Anupam Mitra, Aaron Szasz, Akel Hashim, Justin Kalloor, Wibe Albert de Jong, Costin Iancu, and Ed Younis. 2024. Ac/dc: Automated compilation for dynamic circuits. *arXiv preprint arXiv:2412.07969* (2024).

[26] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. 2020. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research* 2, 4 (Dec 2020).

[27] Francisco JR Ruiz, Tuomas Laakkonen, Johannes Bausch, Matej Balog, Mohammadamin Barekatain, Francisco JH Heras, Alexander Novikov, Nathan Fitzpatrick, Bernardino Romera-Paredes, John Van De Wetering, et al. 2025. Quantum circuit optimization with alphatensor. *Nature Machine Intelligence* 7, 3 (2025), 374–385.

[28] Seetharami Seelam, Jerry M Chow, Antonio Córcoles, Sarah Sheldon, Tushar Mittal, Abhinav Kandala, Sean Dague, Ian Hincks, Hiroshi Horii, Blake Johnson, et al. 2026. Reference architecture of a quantum-centric supercomputer. *arXiv preprint arXiv:2603.10970* (2026).

[29] Sumeet Shirgure, Efekan Kökcü, Anupam Mitra, Wibe Albert de Jong, Costin Iancu, and Siyuan Niu. 2026. Characterizing and Benchmarking Dynamic Quantum Circuits. *arXiv preprint arXiv:2604.03360* (2026).

[30] Sagar Silva Pratapsi, Patrick H Huber, Patrick Barthel, Sougato Bose, Christof Wunderlich, and Yasser Omar. 2023. Classical half-adder using trapped-ion quantum bits: Toward energy-efficient computation. *Applied Physics Letters* 123, 15 (2023).

[31] Adrien Suau, Yiming Zhang, Purva Thakre, Yilun Zhao, Kabir Dubey, Jose A. Bolanos, Arabella Schelpe, Tianyi Hao, Philip Seitz, Gian Giacomo Guerreschi, Ángela Elisa Álvarez Pérez, Reinhard Stahn, Jerome Lenssen, Brendan Reid, and Austin Fowler. 2026. tqec: A Python package for topological quantum error correction. *Journal of Open Source Software* 11, 120 (2026), 9142. doi:10.21105/joss.09142

[32] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. 2017. Error Mitigation for Short-Depth Quantum Circuits. *Physical Review Letters* 119, 18, Article 180509 (Nov. 2017), 180509 pages.

[33] Barbara M Terhal. 2015. Quantum error correction for quantum memories. *Reviews of Modern Physics* 87, 2 (2015), 307–346.

[34] The Green Grid. 2014. *A Power Measurement Methodology for Large-Scale, High-Performance Computing*. Technical Report.

[35] Yue Wu, Namitha Liyanage, and Lin Zhong. 2025. Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 639–654. doi:10.1145/3676641.3716005

[36] Yanzhang Zhu, Chen-Yu Peng, Yun Hao Chen, Yeong-Luh Ueng, and Di Wu. 2026. Lottery BP: Unlocking Quantum Error Decoding at Scale. *arXiv* (2026).

[37] Hang Zou, Martin Rahm, Anton Frisk Kockum, and Simon Olsson. 2025. Generative flow-based warm start of the variational quantum eigensolver. *npj Quantum Information* (2025).